

Lecture: 0011

Combinational Building Blocks – II

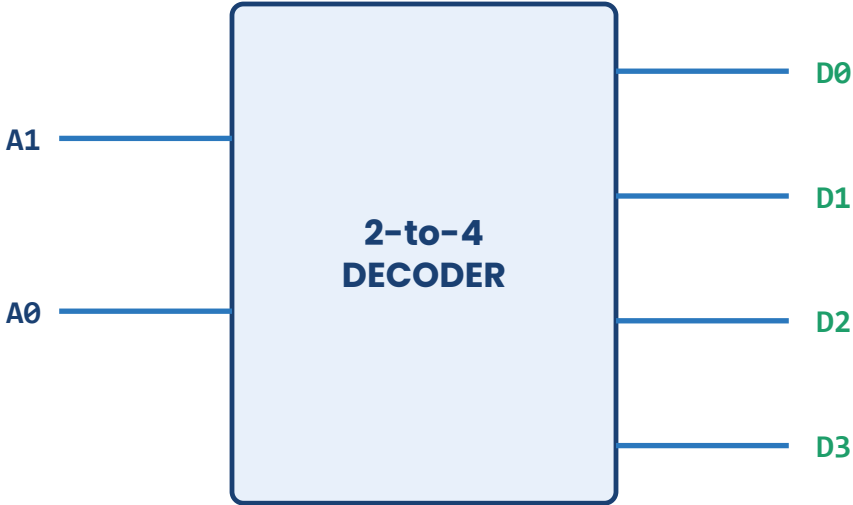
Two Philosophies for a CPU's Language

*The instruction set is the contract between hardware and software —
and there are two great ways to write it.*

CSA222: Modern Computer Architecture

The n -to- 2^n Decoder

Turn an n -bit binary code into exactly one active output line.



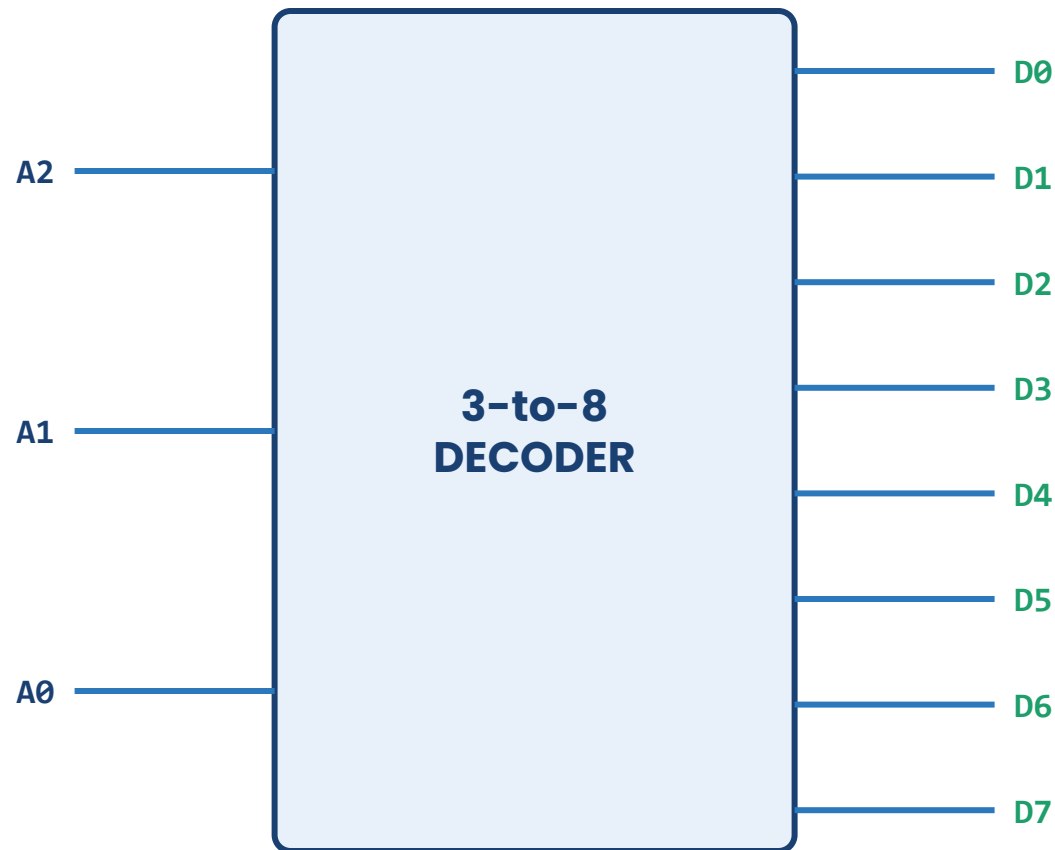
INPUT CODE → ONE HOT OUTPUT

A1	A0	D3	D2	D1	D0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Exactly one output is 1 at a time — this is called 'one-hot' encoding. Each output is a single minterm of the input code.

Block Diagram: 3-to-8 Decoder

Routing a 3-bit binary input code to exactly one of eight unique active-high output lines.



THE SCHEMATIC MODEL

Converts an n -bit coded input into a 2^n one-hot output selection.

Perfect spatial mapping: any single binary combination forces exactly one output line high.

Specifications

Inputs: 3 address bits

Outputs: 8 distinct minterms

Truth Table: 3-to-8 Decoder

The 3-bit binary input code activates exactly one of the eight minterm output lines (one-hot).

A2	A1	A0	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

THE DECISION MATRIX

Every 3-bit input pattern generates a unique minterm value.

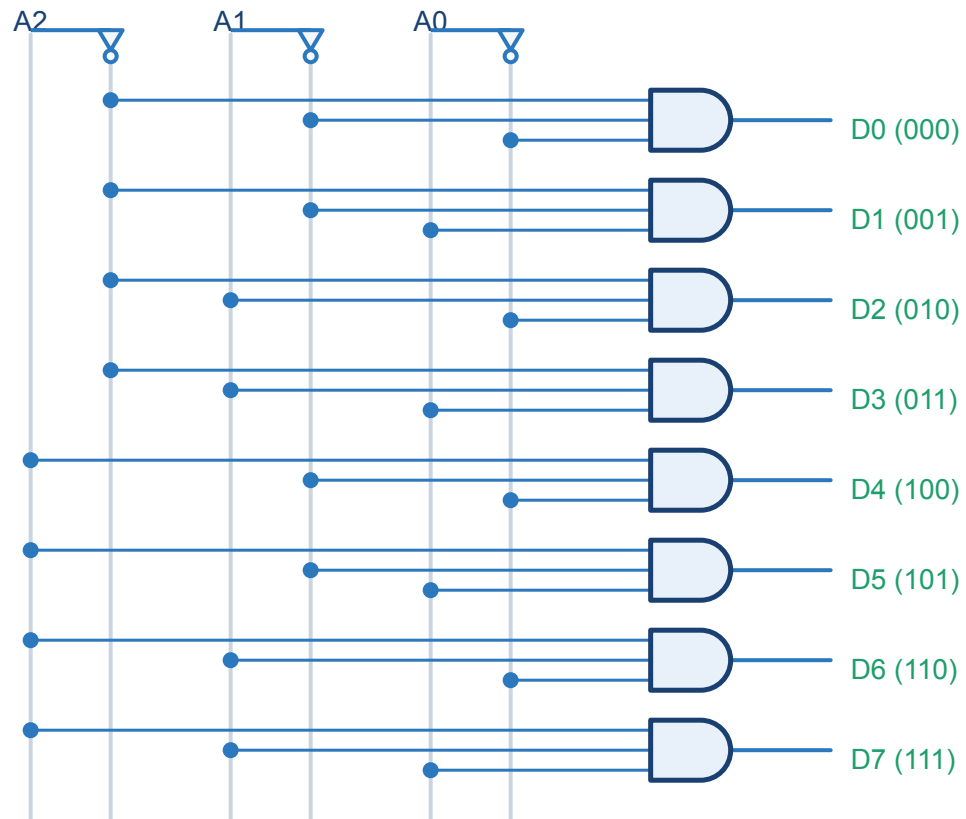
Only one output line "fires" (equals 1) at any given time.

This is the mathematical definition of a one-hot representation.

Inputs = 3 bits (A2, A1, A0)
Outputs = 8 lines (D0 to D7)

Inside a 3-to-8 Decoder

Each output is one AND gate computing a single minterm of the 3-bit input.



THE PATTERN

A 3-to-8 decoder uses 8 AND gates + 3 inverters.

Output D_k fires only for the one input code equal to k .

Generalises: an n -to- 2^n decoder needs 2^n AND gates — the cost grows exponentially with address width.

MUX vs DEMUX vs Decoder

Three cousins — know which one a problem is asking for.

	MULTIPLEXER	DEMULTIPLEXER	DECODER
Inputs	2^n data + n select	1 data + n select	n code bits (+enable)
Outputs	1	2^n	2^n (one-hot)
Does what	Selects one input	Routes to one output	Activates one line per code
One-liner	Many $\rightarrow$ one	One $\rightarrow$ many	Code $\rightarrow$ one-hot

Memory hook: a MUX and a DEMUX are mirror images; a decoder is a DEMUX whose 'data' input is just a constant 1 (an enable).

Quick Check

Show of hands — which block does the job?

Q. "Select one of 8 sensor readings to send to the CPU." Which block?

A**8:1 MUX**

✓ *many inputs → one output*

B**1:8 DEMUX****C****3-to-8 Decoder****D****Full Adder**

Trap: a decoder activates a line from a CODE — it doesn't pass data through. This is a MUX job.

03

Adders

The arithmetic core — from adding two bits to adding two 32-bit numbers.

STAGE 1: TWO BITS (HALF / FULL ADDER)

Inputs: A, B → Sum, Carry

The basic building blocks of binary addition. Handles single-digit inputs.

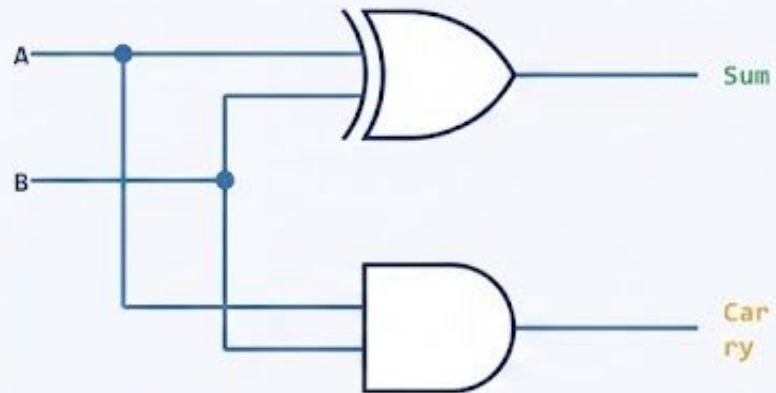
STAGE 2: Multi-bit (32-BIT ADDER)

Inputs: A[31:0], B[31:0] → Sum[31:0]

Chaining adders together to compute complex, multi-word arithmetic.

The Half Adder

Adds two single bits. Produces a Sum and a Carry — but can't accept a carry in.



TRUTH TABLE

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

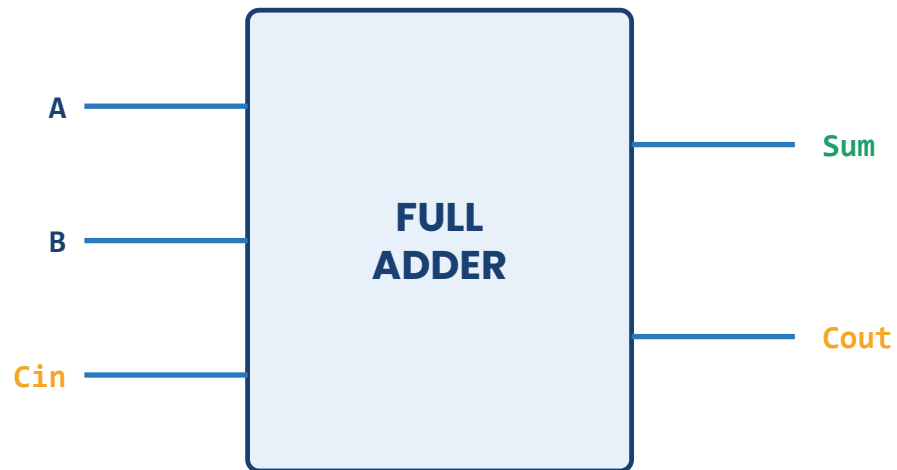
$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

"Half" because it can't take a carry-IN from a previous stage.

The Full Adder

Adds THREE bits: A, B, and a carry-in. This is the real workhorse.



TRUTH TABLE

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

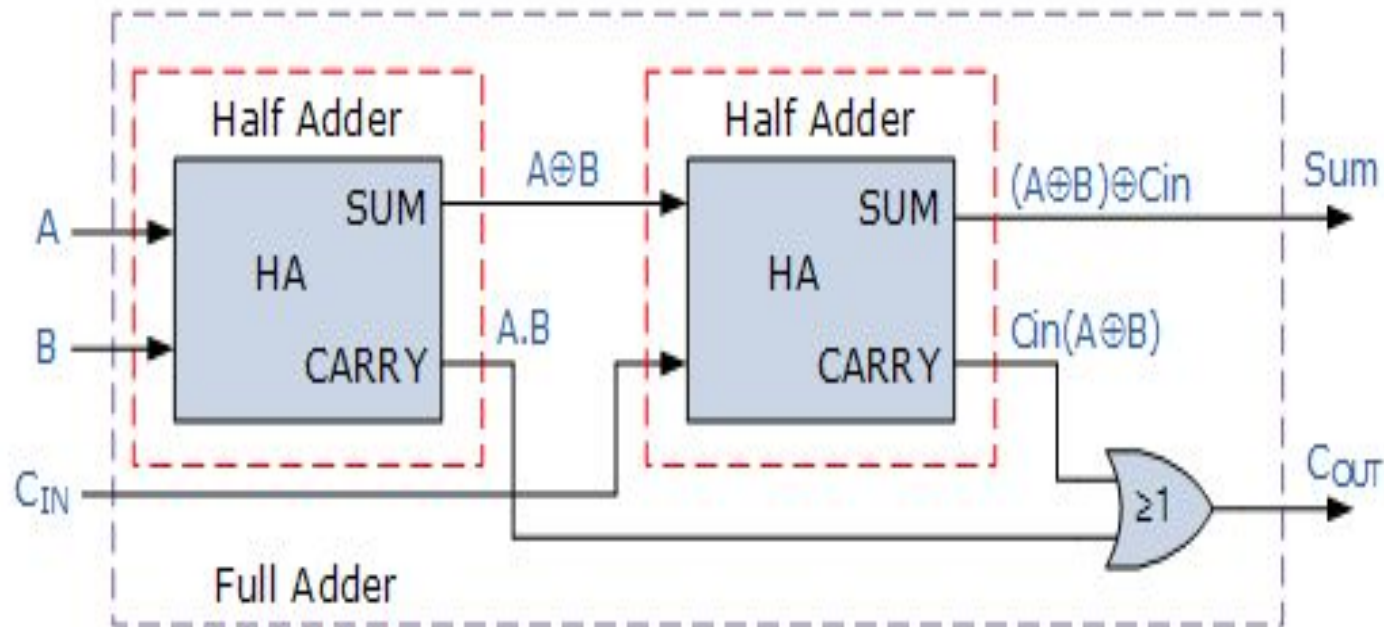
EQUATIONS

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + \text{Cin}(A \oplus B)$$

Full Adder = Two Half Adders + an OR

Compose the block you already have to build the bigger one.



THE IDEA

HA1 adds $A + B$.

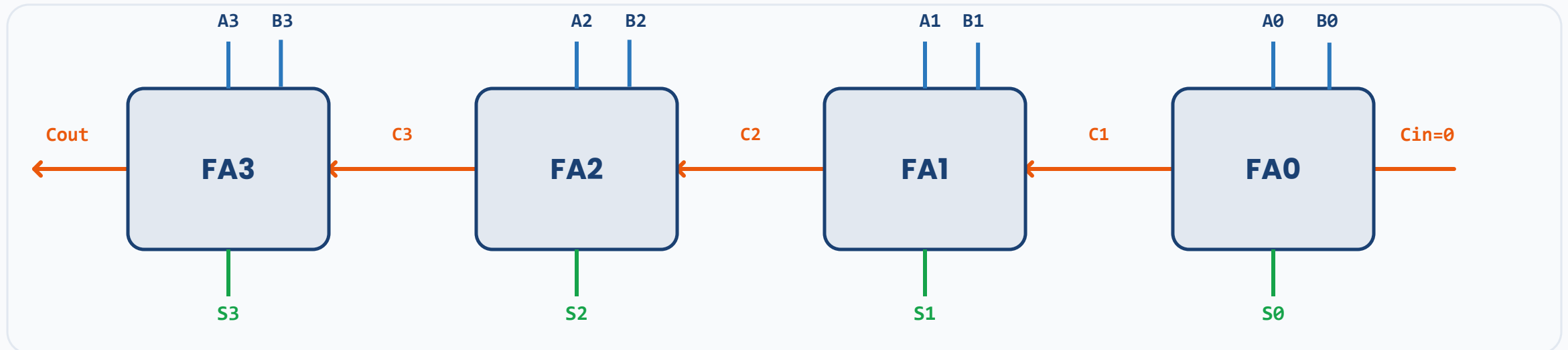
HA2 adds that partial sum + C_{IN} .

Either half can generate a carry, so an OR merges them into C_{OUT} .

Same lesson as MUX cascading: reuse a proven block.

Ripple-Carry Adder

Chain full adders — each one's carry-out feeds the next one's carry-in.



The Carry Ripple Effect

Carry ripples right → left, one full adder at a time.

Just like adding by hand, carrying the 1. Each block's carry-out feeds directly into the next stage's carry-in.

Scaling & Complexity

A 4-bit adder = 4 full adders. A 32-bit adder = 32 chained.

Simple and straightforward to build — but look out for carry propagation delay (watch the timing on next slide).

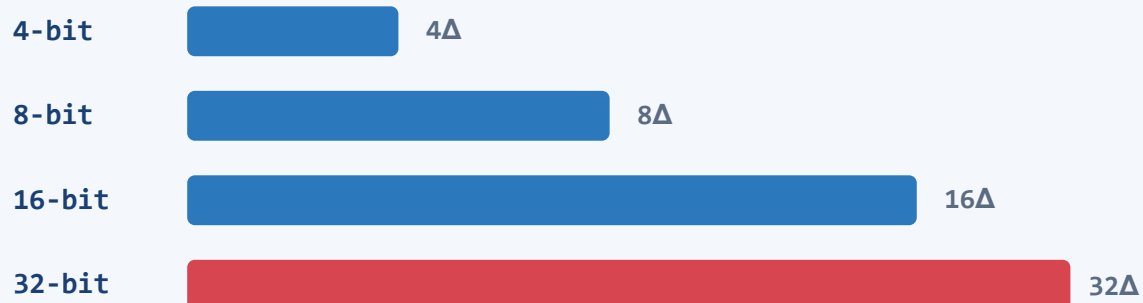
The Catch: Carry Propagation Delay

The last bit can't settle until the carry has rippled through every stage.

THE BOTTLENECK

Each full adder must wait for the carry from the one before it. For a 32-bit ripple adder, the worst-case delay is ~ 32 gate-delays stacked end to end.

Delay grows linearly with width:



THE FIX (PREVIEW)

Carry-Lookahead Adders

compute all carries in parallel from the inputs directly — instead of waiting for them to ripple.

This trades more gates (area) for far less delay (speed) — the classic area-vs-speed tradeoff from L2, now real.

Every GHz CPU uses lookahead or hybrid adders — ripple-carry is far too slow at modern clock speeds.

Common Mistakes to Avoid

The slips that cost marks in exams and interviews.



Confusing MUX and DEMUX

MUX = many→one (select an input). DEMUX = one→many (route to an output). Opposite directions.



Forgetting the carry-in

A half adder can't chain — it has no Cin. Multi-bit addition needs FULL adders.



Mixing up DEMUX and decoder

A decoder is a DEMUX with its data input tied to 1 — it activates a line, it doesn't pass data.



Ripple adder is 'fast'

It's simple, not fast. Delay grows linearly with width — real CPUs use carry-lookahead.



Wrong select-line count

2^n inputs need n select lines. An 8:1 MUX needs 3 selects, not 8. Count carefully.



MUX method: too many selects

A 4:1 MUX handles a 3-variable function (2 selects + data tricks) — don't use 3 selects.

Key Takeaways

Five sentences. These blocks are the alphabet of every processor.

- 01 A MUX selects one of many inputs ($Y = S'A + SB$ for 2:1); a DEMUX does the reverse — one input to one of many outputs.
- 02 A decoder turns an n -bit code into one active line out of 2^n — the foundation of all addressing.
- 03 Any function can be built from a decoder + OR, or from a single MUX with cleverly wired data inputs (the FPGA principle).
- 04 A half adder adds two bits; a full adder adds three (with carry-in) and chains into multi-bit ripple adders.
- 05 Ripple-carry is simple but slow — carry propagation delay grows with width, which is why real CPUs use carry-lookahead.

Practice & Homework

Build and verify — the only way this material sticks.

01

MUX from Gates

Build a 2:1 MUX using AND, OR, NOT gates. Verify with a truth table for all S, A, B.

02

Decoder Decode

Fill the full output table of a 3-to-8 decoder for all 8 input codes.

03

HA → FA

Build a half adder, then extend to a full adder. Identify the extra signal it needs and why.

EXIT TICKET

1. Implement a 4:1 MUX using only 2:1 MUXes.
2. Use a 4:1 MUX to implement $F(A,B,C) = \Sigma m(1,3,5,6)$.
3. A 3-to-8 decoder: how many AND gates does it use, and what does each compute?

Rapid-Fire Recap

Three questions — shout the answers.

Q1**A 2:1 MUX selects...**

one of two inputs

Q2**The Sum output of a half adder is...**

A XOR B

Q3**A 3-to-8 decoder activates...**

exactly one of 8 outputs

If all three were instant, you're ready for Lab work and L4 (Number Systems & Two's Complement).

Go Deeper

References and where to practise before the next lab.

Harris & Harris

Digital Design and Computer Architecture — Chapter 2 (Combinational Logic Design). The primary text for this module.

Patterson & Hennessy

Computer Organization and Design — Appendix on logic; connects these blocks to the datapath in Module 4.

Simulate it

Build every block in Logisim or on the Verilog/GTKWave flow from the labs. Seeing a carry ripple in a waveform makes Slide 23 click.

END OF L03

From blocks, a processor.

NEXT — L04

Number Systems, Two's Complement & Overflow — how the adders you just built actually represent negative numbers, and when they silently go wrong.



**Thanks
for
watching!**